

Pandas入門

DataFrame ・ *Series* ・ *index* ・ *columns*

rkskmt

1

NumPyの表に名前をつける

2

Pandasとは

- Pandasは **表形式データ (tabular data)** を扱うためのライブラリ
 - Excelの表、CSV、実験結果、アンケート結果など
 - データ分析の入口で非常によく使う
- NumPyの **ndarray** と同じく、Pythonの標準機能ではない

ターミナルからインストール：

Shell

```
$ pip install pandas
```

冒頭で import (**pd** としてインポートするのが一般的)：

Code

```
1 import pandas as pd
```

3

今回のゴール

- **DataFrame / Series / index / columns** を理解する
- 集計結果を **matplotlibで可視化** できるようにする

4

質問：田中さんの数学Iの点は？

- 2人の生徒（田中・佐藤）の3科目（現代文・数学I・英会話）の点数がある

Code

```
1 import numpy as np
2
3 arr = np.array([
4     [80, 70, 90],
5     [65, 75, 85],
6 ])
```

- 「田中さんの数学Iの点」を取り出すコードを書け

❗ 答え：arr[0, 1] で合ってる？自信ある？

- 田中さんが0行目、数学Iが1列目という情報は **配列のどこにも書いていない**
- 取り出した **70** も、画面に出るのはただの数字 — **何の70点か分からない**
- 計算はNumPyで十分。困るのは **意味（名前）の管理**
- 表に **名前** をつけたもの、それが今日の **DataFrame**

DataFrameとは

- Pandasの主役となる、**名前付きの表（テーブル, table）**
- **データ** + 行の名前（**index**）+ 列の名前（**columns**）の3点セット
- 行・列に名前をつけて表を扱う考え方は、Excelやデータベースなど多くの道具に共通

	現代文	数学I	英会話
田中	80	70	90
佐藤	65	75	85

上端の見出し（現代文・数学I・英会話）が **columns**、左端の見出し（田中・佐藤）が **index**

💡 **pandasの基本はDataFrameなので用語を覚えよう**

DataFrame = データ + index（行の名前）+ columns（列の名前）

ndarrayからDataFrameを作る

- `pd.DataFrame()` に `ndarray` を渡す
- 最初は列名も行名も指定しない

Code

```
1 import numpy as np
2 import pandas as pd
3
4 arr = np.array([
5     [80, 70, 90],
6     [65, 75, 85],
7 ])
8
9 df = pd.DataFrame(arr)
10 print(df)
```

Result

	0	1	2
0	80	70	90
1	65	75	85

DataFrame は表として表示され、左端と上端に **0, 1, 2, ...** の番号が自動でつく

7

indexとcolumnsを確認

- 行の名前を **index**
- 列の名前を **columns** という
- 指定しない場合は、どちらも自動で番号が振られる

Code

```
1 import numpy as np
2 import pandas as pd
3
4 arr = np.array([[80, 70, 90], [65, 75, 85]])
5 df = pd.DataFrame(arr)
6
7 print(df.index)
8 print(df.columns)
```

Result

```
RangeIndex(start=0, stop=2, step=1)
RangeIndex(start=0, stop=3, step=1)
```

① 自動でつく番号

- **RangeIndex(start=0, stop=2, step=1)** は「0から1までの行番号」
- **RangeIndex(start=0, stop=3, step=1)** は「0から2までの列番号」
- Pythonの **range()** と同じく、**stop** の値は含まない

8

columnsを指定する

- 列の意味が分かるように **columns=** を指定する
- これだけで表がかなり読みやすくなる

Code

```
1 import numpy as np
2 import pandas as pd
3
4 arr = np.array([[80, 70, 90], [65, 75, 85]])
5 df = pd.DataFrame(arr, columns=["現代文", "数学I", "英会話"])
6
7 print(df)
```

Result

	現代文	数学I	英会話
0	80	70	90
1	65	75	85

もう **df[1]** ではなく **df["数学I"]**。「何列目か」ではなく「何の列か」で読める

indexも指定する

- 行の意味が分かるように **index=** も指定する
- この例では、行名を生徒名にする

Code

```
1 import numpy as np
2 import pandas as pd
3
4 arr = np.array([[80, 70, 90], [65, 75, 85]])
5 df = pd.DataFrame(arr, columns=["現代文", "数学I", "英会話"], index=["田中", "佐藤"])
6
7 print(df)
```

Result

	現代文	数学I	英会話
田中	80	70	90
佐藤	65	75	85

これで数字の並びに「誰の・何の点か」という意味がついた

名前があると何がうれしい？



- `df["数学I"]` と書くと、数学Iの列だけ取り出せる
- `df.mean()` と書くと、列ごとの平均が計算できる
- 出力にも列名が残るので、結果を読み間違えにくい

Code

```
1 import numpy as np
2 import pandas as pd
3
4 arr = np.array([[80, 70, 90], [65, 75, 85]])
5 df = pd.DataFrame(arr, columns=["現代文", "数学I", "英会話"], index=["田中", "佐藤"])
6
7 print(df.mean())
```

Result

```
現代文    72.5
数学I     72.5
英会話    87.5
dtype: float64
```

💡 名前付きの表が強い

- `axis=0` のような番号指定より読みやすい場面が多い
- データ分析では **名前付きの表 (DataFrame)** が効く

11

Seriesを理解する

12

列を取り出す (Series)

- `df["列名"]` で1列だけ取り出せる
- 取り出した1列は **DataFrame** 型ではなく **Series** 型
- (表は `pd.DataFrame()` にリストを直接渡しても作れる — 以降はこの書き方)

Code

```
1 import pandas as pd
2
3 df = pd.DataFrame([[80, 70, 90], [65, 75, 85]],
4                   columns=["現代文", "数学I", "英会話"], index=["田中", "佐藤"])
5 math = df["数学I"]
6
7 print(math)
8 print(type(df))
9 print(type(math))
```

Result

```
田中    70
佐藤    75
Name: 数学I, dtype: int64
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
```

13

DataFrameとSeries

- DataFrameは **表**
- Seriesは **1列**
- Seriesにも **index** がある

```
1 DataFrame = 表
2 Series    = 1列
```

Code

```
1 import pandas as pd
2
3 df = pd.DataFrame([[80, 70, 90], [65, 75, 85]],
4                   columns=["現代文", "数学I", "英会話"], index=["田中", "佐藤"])
5 math = df["数学I"]
6
7 print(math.index)
8 print(math.values)
```

Result

```
Index(['田中', '佐藤'], dtype='object')
[70 75]
```

`math.index` は生徒名、`math.values` は点数だけのNumPy配列

14

Seriesは「値」と「名前」のセット

- NumPy配列は値だけ
- Seriesは値に **index** がついている

見方	中身
<code>math.index</code>	<code>["田中", "佐藤"]</code>
<code>math.values</code>	<code>[70, 75]</code>

Code

```
1 import pandas as pd
2
3 df = pd.DataFrame([[80, 70, 90], [65, 75, 85]],
4                   columns=["現代文", "数学I", "英会話"], index=["田中", "佐藤"])
5 math = df["数学I"]
6
7 print(math.mean())
```

Result

```
|
| 72.5
|
```

1列（Series）だけでも、平均・最大・最小などの計算ができる

列名で取り出す練習

- 現代文の列を取り出す
- 英会話の列を取り出す
- それぞれの平均を計算する

Code

```
1 import pandas as pd
2
3 df = pd.DataFrame([[80, 70, 90], [65, 75, 85]],
4                   columns=["現代文", "数学I", "英会話"], index=["田中", "佐藤"])
5
6 japanese = df["現代文"]
7 english = df["英会話"]
8
9 print(japanese.mean())
10 print(english.mean())
```

Result

```
72.5
87.5
```

💡 まずは列単位で考える

- Pandasでは「列を取り出す → 計算する」が基本
- 1列を取り出すと **Series**
- 複数列を取り出すと **DataFrame**

16

演習：型を見失わない



田中・佐藤の **df** で、次の4つが **DataFrame** / **Series** / **ndarray** / 数値のどれになるかを **実行する前に予想** し、**type(...)** で確かめよ（**Series** になったものは **.index** も表示）。

1. **df**
2. **df["数学I"]**（1列）
3. **df["数学I"].values**（値だけ）
4. **df["数学I"].mean()**（集計）

▶ 答え合わせ（df を作って type を確認）

💡 操作のたびに自問する

- いま手元は **DataFrame** か **Series** か（あるいは値か）
- その **index** / **columns** は何か

17

CSVとは

- CSVは表をテキストで保存する形式
 - **C**omma **S**eparated **V**alues
 - カンマで値を区切る
- ExcelやGoogleスプレッドシートからも書き出せる

```
1 名前,クラス,学年,現代文,数学I,英会話,物理,欠席日数
2 田中,A,1,80,70,90,85,2
3 佐藤,B,1,65,75,85,72,4
4 鈴木,A,2,88,91,84,90,1
```

- 1行目は列名として扱われることが多い
- 列が増えるほど、Pandasで扱う意味が出てくる
- Pandasでは `pd.read_csv()` で読み込む

CSVを読み込む

1. [data/students.csv](#) をダウンロード
2. 作業フォルダに `data` フォルダを作る
3. その中に `students.csv` を置く
4. Pythonから `data/students.csv` で読み込む
 - ※ `students.csv` は授業用の **架空データ**（実在の個人ではない）

```
Code
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4 print(df.head())
```

	名前	クラス	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	A	1	80	70	90	85	2
1	佐藤	B	1	65	75	85	72	4
2	鈴木	A	2	88	91	84	90	1
3	高橋	C	2	72	68	76	80	3
4	伊藤	A	1	58	57	62	65	6

CSVの1行目が `columns` になり、`index` は指定しなければ自動で番号が振られる

❗ ここから `df` は `students.csv` に変わる

- ここまでの田中・佐藤（2人×3科目）は **練習用の小さな表**
- 以降の `df` は **students.csv（20人×8列）** — 集計・可視化はこのデータで進める

読み込んだ表の形を確認

- データを読み込んだら、まず **中身と形** を確認する
- head()** は先頭数行を表示する
- shape** は **(行数, 列数)**

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4
5 print(df.head())    # 先頭5行 (中身)
6 print(df.shape)     # (行数, 列数)
7 print(df.columns)   # 列名
8 print(df.index)     # 行のラベル
```

Result

	名前	クラス	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	A	1	80	70	90	85	2
1	佐藤	B	1	65	75	85	72	4
2	鈴木	A	2	88	91	84	90	1
3	高橋	C	2	72	68	76	80	3
4	伊藤	A	1	58	57	62	65	6

(20, 8)
Index(['名前', 'クラス', '学年', '現代文', '数学I', '英会話', '物理', '欠席日数'],
dtype='object')
RangeIndex(start=0, stop=20, step=1)

名前をindexにする読み込み

- index_col="名前"** とすると、名前列をindexにできる
- 「行名として使いたい列」があるときに使う

Code

```
1 import pandas as pd
2
3 df_name = pd.read_csv("data/students.csv", index_col="名前")
4 print(df_name.head())
```

Result

	クラス	学年	現代文	数学I	英会話	物理	欠席日数
名前							
田中	A	1	80	70	90	85	2
佐藤	B	1	65	75	85	72	4
鈴木	A	2	88	91	84	90	1
高橋	C	2	72	68	76	80	3
伊藤	A	1	58	57	62	65	6

💡 まずはここだけ分かれば十分

- 最初は自動番号のindexのままで問題ない
- indexとは行名** — それが分かればOK

単一列を選択する（おさらい）

- **df["列名"]** で1列取り出すと **Series** — 集計の入口としてよく使う

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4 math = df["数学I"]
5
6 print(math.head()) # 20人分あるので先頭5行だけ
```

Result

0	70
1	75
2	91
3	68
4	57
Name: 数学I, dtype: int64	

複数列を選択する

- **df[["列名1", "列名2"]]** のように列名の **リスト** を渡す（角括弧が2重）
- 返ってくるのは **Series** ではなく **DataFrame**

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4 cols = df[["名前", "クラス", "数学I", "英会話"]]
5
6 print(cols.head())
```

Result

	名前	クラス	数学I	英会話
0	田中	A	70	90
1	佐藤	B	75	85
2	鈴木	A	91	84
3	高橋	C	68	76
4	伊藤	A	57	62

条件式は「残す行の印」を作る

- 列に条件式を書くと、各行について **True** / **False** が返る

```
Code
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4
5 print((df["数学I"] >= 80).head()) # 先頭5行
```

```
Result
0    False
1    False
2     True
3    False
4    False
Name: 数学I, dtype: bool
```

数学I が80点以上の行だけ **True** — これが「どの行を残すか」の印になる

条件抽出 (filtering)

- 条件式を **df[...]** に入れると、**True** の行だけが残る

```
Code
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4
5 print(df[df["数学I"] >= 80].head()) # 先頭5行
```

```
Result
   名前 クラス  学年  現代文  数学I  英会話  物理  欠席日数
2   鈴木     A    2    88    91    84   90     1
5   渡辺     B    3    84    88    78   86     2
9   加藤     C    3    90    95    88   92     0
10  吉田     A    3    83    86    91   89     1
12  佐々木    C    1    74    80    69   83     2
```

数学I が80点以上の行だけが残る（20人中9人が該当）

文字列の条件抽出

- 数値だけでなく、文字列でも条件を作れる（**==** は「等しい」）

```
Code
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4
5 print(df[df["クラス"] == "A"])
```

	名前	クラス	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	A	1	80	70	90	85	2
2	鈴木	A	2	88	91	84	90	1
4	伊藤	A	1	58	57	62	65	6
7	中村	A	2	76	65	82	74	3
10	吉田	A	3	83	86	91	89	1
13	山口	A	2	62	78	75	70	5
16	木村	A	1	92	89	94	91	0
19	斎藤	A	3	77	93	85	90	1

クラス が **"A"** の行だけが残る（名簿・アンケートでよく使う）

28

複数条件で抽出する

- 複数条件は **&** でつなぐ
- それぞれの条件を丸括弧で囲む

```
Code
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4
5 print(df[
6     (df["クラス"] == "A")
7     & (df["数学I"] >= 80)
8 ])
```

	名前	クラス	学年	現代文	数学I	英会話	物理	欠席日数
2	鈴木	A	2	88	91	84	90	1
10	吉田	A	3	83	86	91	89	1
16	木村	A	1	92	89	94	91	0
19	斎藤	A	3	77	93	85	90	1

⚠ and ではなく &

- Pandasの条件抽出では **and** ではなく **&**
- 条件ごとに丸括弧をつける
- この書き方は最初だけ少し変だが、よく使う

29



- 前ページのコード を1か所ずつ変えて実行

- 1 条件式だけ実行する（**df[...]** を外す）→ 各行に True/False が返る
- 2 **&** を **|** にする（「両方」→「どちらか一方」で残る行が増える）
- 3 数学Iの基準を反転する（80点未満）
- 4 英会話も80点以上、と条件を足す

30

列を追加する

- **df["新しい列名"] = ...** で列を追加できる
- 既存の列を使って、新しい列を計算することが多い

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4 subjects = ["現代文", "数学I", "英会話", "物理"]
5
6 df["平均点"] = df[subjects].mean(axis=1)
7
8 print(df.head())
```

Result

	名前	クラス	学年	現代文	数学I	英会話	物理	欠席日数	平均点
0	田中	A	1	80	70	90	85	2	81.25
1	佐藤	B	1	65	75	85	72	4	74.25
2	鈴木	A	2	88	91	84	90	1	88.25
3	高橋	C	2	72	68	76	80	3	74.00
4	伊藤	A	1	58	57	62	65	6	60.50

平均点 は4科目（現代文・数学I・英会話・物理）の平均

① NumPyと同じ感覚

- **mean(axis=1)** は、行ごとに平均を計算する（いま作った **平均点**）
- **df["数学I"] * 1.1** のように、列全体へまとめて計算もできる
- for文を書かなくても列全体に計算がかかる

31

平均点列を集計する

- 4科目から作った **平均点** 列を **Series** として取り出す
- Seriesには平均・最大・最小など **集計 (aggregation)** のメソッドがある

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4 subjects = ["現代文", "数学I", "英会話", "物理"]
5 df["平均点"] = df[subjects].mean(axis=1)
6 scores = df["平均点"]
7
8 print(scores.mean())
9 print(scores.max())
10 print(scores.min())
```

Result

```
76.7875
91.5
56.75
```

i CSVを読んだあとの典型的な流れ

- 1 CSVを読む → 必要な列を作る → 集計する → 可視化する

33

クラスごとに人数を数える

- **groupby("クラス")** で、クラスごとに行をグループへ分ける
- グループの中身を見るには **.size()** などをつける必要がある

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4
5 print(df.groupby("クラス").size())
```

Result

```
クラス
A      8
B      6
C      6
dtype: int64
```

Aが8人・Bが6人・Cが6人で合計20人 — 全員がどこかのグループに分けられている

34

クラスごとに平均点を出す

- `groupby("クラス")` に、集計する列と方法を続ける
- `["平均点"]` で列を選び、`.mean()` でクラスごとの平均を出す

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4 subjects = ["現代文", "数学I", "英会話", "物理"]
5 df["平均点"] = df[subjects].mean(axis=1)
6
7 avg = df.groupby("クラス")["平均点"].mean()
8 print(avg)
```

Result

```
クラス
A    80.062500
B    71.625000
C    77.583333
Name: 平均点, dtype: float64
```

CSVの **クラス** 列と、追加した **平均点** 列を使う。結果は **DataFrame** ではなく **Series**

groupbyの結果を確認

- groupbyの結果も **Series**
- **index** がクラス名
- **values** が平均点

Code

```
1 import pandas as pd
2
3 df = pd.read_csv("data/students.csv")
4 subjects = ["現代文", "数学I", "英会話", "物理"]
5 df["平均点"] = df[subjects].mean(axis=1)
6 avg = df.groupby("クラス")["平均点"].mean()
7
8 print(type(avg))
9 print(avg.index)
10 print(avg.values)
```

Result

```
<class 'pandas.core.series.Series'>
Index(['A', 'B', 'C'], dtype='object', name='クラス')
[80.0625      71.625      77.58333333]
```

💡 groupbyの結果

- 1 groupby の結果も Series
- 2 index が「クラス」
- 3 values が「平均点」

Tweak

groupby は「分ける × 集計する列 × 集計方法」



- さっきのコード を1か所ずつ変えて実行

1

学年でgroupbyする

2

集計する列を欠席日数にする

3

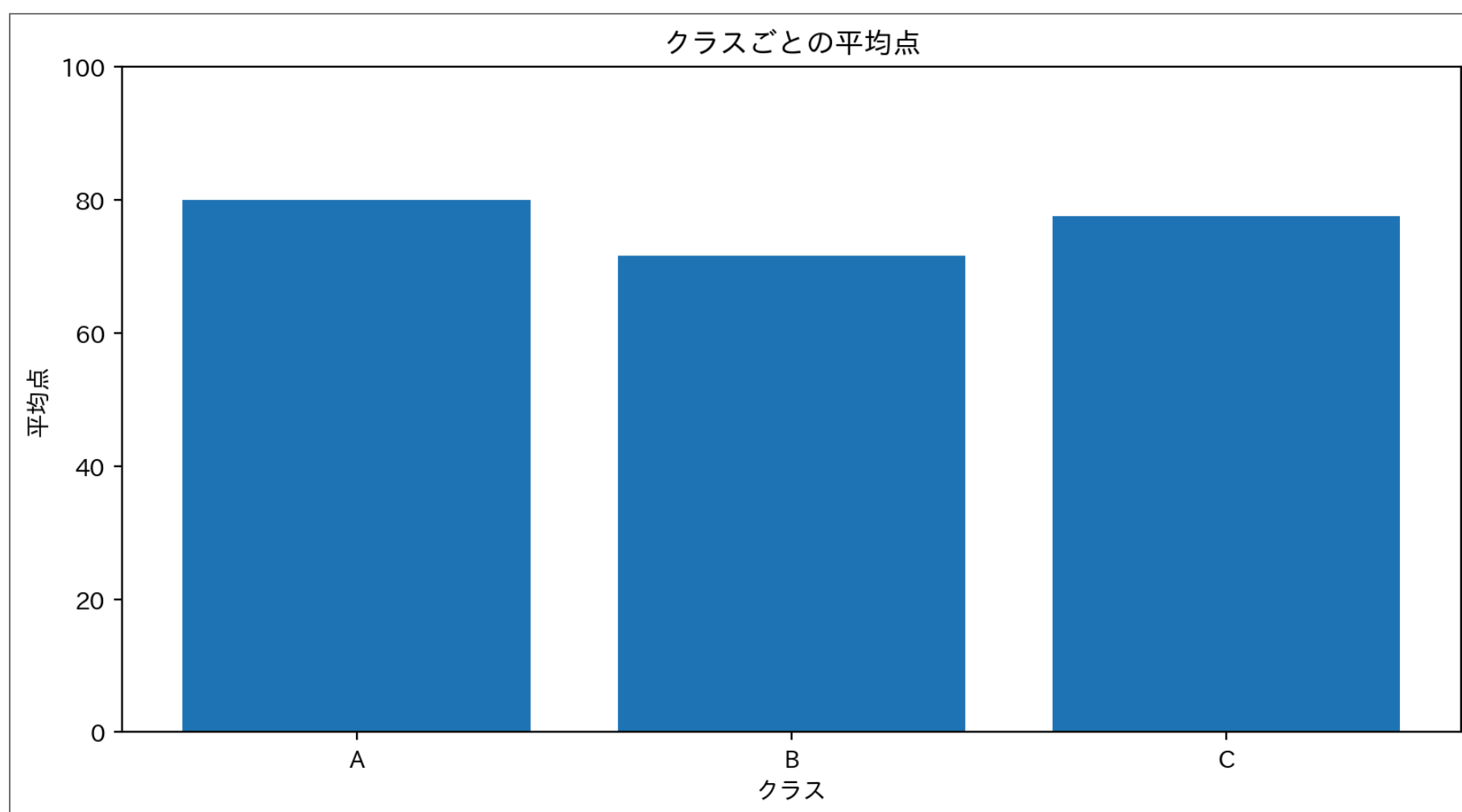
集計を最大値にする

matplotlibで棒グラフにする

- **groupby()** で作った **avg** (Series) を棒グラフにする
- **Series** を **index** と **values** に分けて matplotlib に渡す
- 縦軸は **ylim(0, 100)** で0点～満点に固定（棒グラフは0から見せる）

Code

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 df = pd.read_csv("data/students.csv")
6 subjects = ["現代文", "数学I", "英会話", "物理"]
7 df["平均点"] = df[subjects].mean(axis=1)
8 avg = df.groupby("クラス")["平均点"].mean()
9
10 plt.bar(avg.index, avg.values)
11 plt.xlabel("クラス")
12 plt.ylabel("平均点")
13 plt.title("クラスごとの平均点")
14 plt.ylim(0, 100)
15 plt.show()
```



💡 Pandasからmatplotlibへ

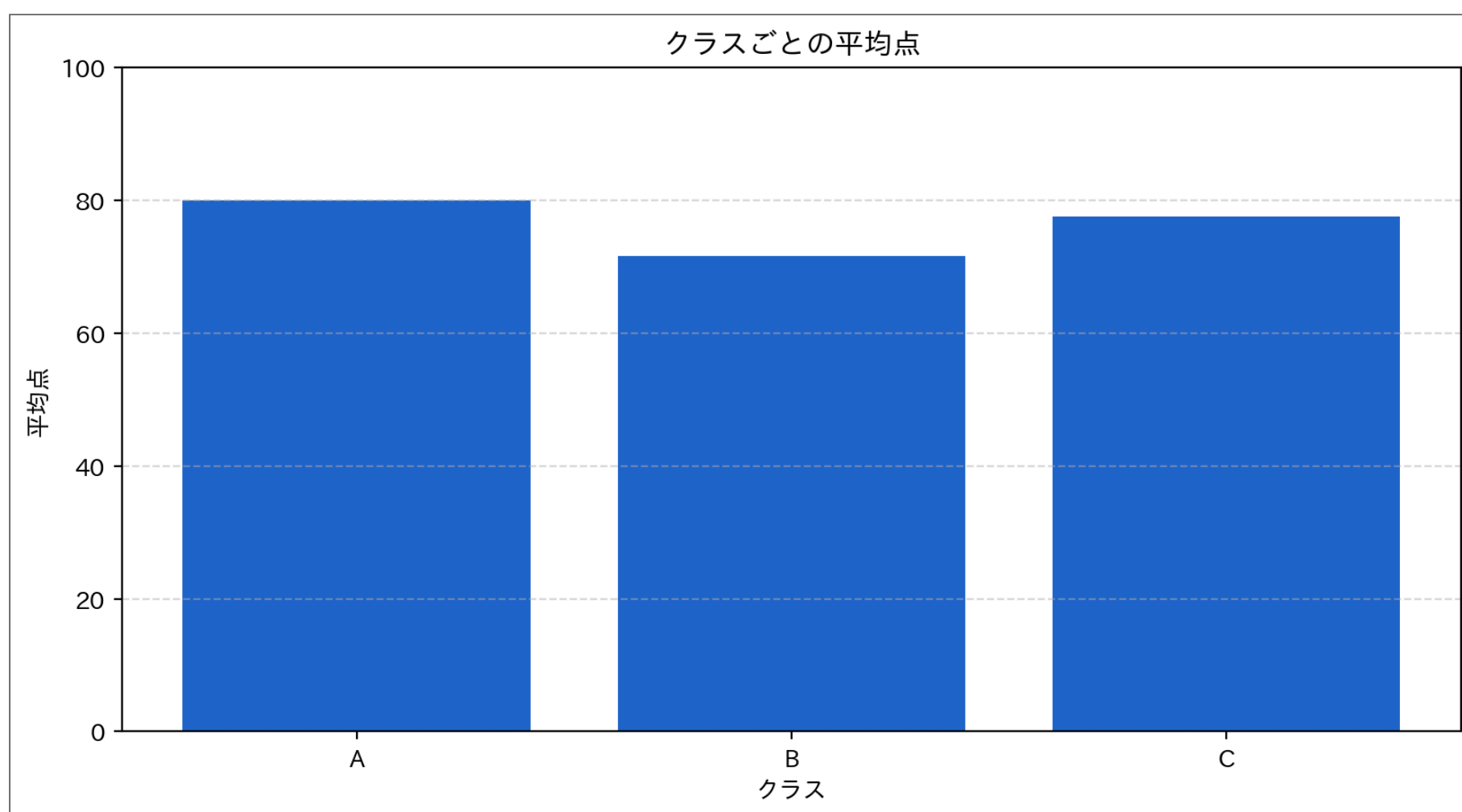
```
1 Series → index / values → matplotlib
```

少し整える

- 色（**color=**）とグリッド（**grid()**）を足すと、比較しやすくなる
- grid(axis="y", ...)** は横線だけを薄く引く

Code

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 df = pd.read_csv("data/students.csv")
6 subjects = ["現代文", "数学I", "英会話", "物理"]
7 df["平均点"] = df[subjects].mean(axis=1)
8 avg = df.groupby("クラス")["平均点"].mean()
9
10 plt.bar(avg.index, avg.values, color="#2266CC")
11 plt.xlabel("クラス")
12 plt.ylabel("平均点")
13 plt.title("クラスごとの平均点")
14 plt.ylim(0, 100)
15 plt.grid(axis="y", linestyle="--", alpha=0.5)
16 plt.show()
```



39

演習：高得点者を可視化



students.csv の20人から **4科目平均が80点以上の高得点者** だけを抜き出し、棒グラフで比較せよ。

- data/students.csv** を読み込み、4科目の平均点の列を追加
- 平均点が80点以上の生徒だけを条件抽出
- 生徒名を横軸、平均点を縦軸に棒グラフを描く（y軸は **0** ～ **100** に固定）

▶ 解答例を見る

40

まとめ

- **DataFrame** は 表
→ データ + **index** + **columns**
- **Series** は 1列
→ 値 + **index**
- **pd.read_csv()** でCSVを読み込める
- 条件抽出は **df[条件]**
- **groupby()** でグループごとの集計ができる
- 集計結果の **index** / **values** を matplotlib に渡して可視化できる

41

演習：実データで明治の東京 vs 21世紀の東京



気象庁が公開している東京150年分の年平均気温をCSVで読み込み、**明治の東京と21世紀の東京は平均で何度ちがうのか**を自分のコードで確かめよ。

0. **tokyo_temp_annual.csv** をダウンロードして **data** フォルダに置く
1. **pd.read_csv()** で読み込み、**columns** で列名を確認
2. 条件抽出で **明治（1911年以前）** と **21世紀（2000年以降）** の行に分ける
3. それぞれの気温の平均を比較（**差分値も表示**）

▶ 解答例を見る

💡 列名で読み書きできる強さ

- 150行のデータでも、**絞り込み1行+平均1行**で答えが出る
- NumPyの「何列目が年だっけ？」が、Pandasでは **列名** で済む

42